

Self-RAG-Inspired Retrieval-Augmented Generation for Vietnamese Legal Question Answering

Vũ Đức Mạnh - 250201075

Nguyễn Hoàng Nam - 250201018

Trường Đại học Công nghệ Thông tin, ĐHQG-HCM

CS2308.CH203 – Chuyên đề nghiên cứu và ứng dụng về Xử lý ngôn ngữ tự nhiên

Tóm tắt nội dung

Mô hình ngôn ngữ lớn (LLM) thường xuyên “ảo giác” (hallucinate) khi trả lời các câu hỏi đòi hỏi kiến thức chuyên ngành chính xác, có thể trích dẫn được – một vấn đề nghiêm trọng trong hỏi-đáp pháp luật, nơi một trích dẫn điều luật sai có thể gây hậu quả thực tế. Sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation – RAG) giúp neo câu trả lời vào bằng chứng được truy xuất, nhưng một pipeline truy xuất-rồi-sinh với số lượng đoạn cố định (top- k) không có cơ chế nào để loại bỏ bằng chứng không liên quan hay kiểm chứng xem câu trả lời sinh ra có thực sự được bằng chứng đó hỗ trợ hay không. Self-RAG (Asai et al., 2024) giải quyết vấn đề này bằng cách huấn luyện một mô hình sinh ra các *token phản tư* (reflection tokens) quyết định khi nào cần truy xuất và tự phê bình đầu ra của chính nó, nhưng cách làm này đòi hỏi phải tinh chỉnh (fine-tune) cả một generator lẫn một critic. Trong đồ án này, chúng tôi xây dựng một hệ thống *lấy cảm hứng từ Self-RAG* (Self-RAG-inspired) tái tạo đúng bốn tín hiệu phản tư nói trên (RETRIEVE, ISREL, ISSUP, ISUSE) hoàn toàn thông qua prompting có cấu trúc trên một LLM instruction-tuned cố định, không tinh chỉnh. Chúng tôi xây dựng và so sánh ba hệ thống – No-RAG, Standard RAG (truy xuất top-5 cố định), và Self-RAG-inspired – trên bộ dữ liệu VLSP2025 DRILL, một benchmark hỏi-đáp pháp luật tiếng Việt thực tế gồm 59.636 điều luật và 2.190 câu hỏi có nhãn. Trên một mẫu đánh giá gồm 187 câu hỏi, hệ Self-RAG-inspired vượt trội cả hai baseline trên mọi chỉ số chất lượng câu trả lời cùng lúc: Correctness (33.2% so với 25.7% / 16.0%), Support (62.0% so với 53.5%), và Usefulness (62.6% so với 25.7% / 50.8%), trong khi bộ lọc liên quan (ISREL) giúp tăng hơn gấp đôi tỷ lệ câu trả lời được hỗ trợ đầy đủ (39.0%→90.3%), một kết quả tái hiện nhất quán qua ba lần đánh giá độc lập. Chúng tôi cũng ghi nhận một phát hiện có phần nghịch lý: Standard RAG lại *kém hữu ích hơn* cả No-RAG, vì nó trung thực từ chối trả lời khi ngữ cảnh cố định của nó không

liên quan, trong khi việc lọc bằng ISREL giải quyết được đánh đổi giữa độ đúng và độ hữu ích này. Chúng tôi cũng trình bày các thất bại kỹ thuật cụ thể khi vận hành một pipeline LLM-judge trên hạ tầng miễn phí, cũng như các hạn chế của một cơ chế phản tư hoàn toàn dựa trên prompting, không tinh chỉnh.

1 Introduction

Mô hình ngôn ngữ lớn (LLM) sinh văn bản trôi chảy nhưng được biết là hay “ảo giác” sự kiện, đặc biệt trong các bối cảnh đòi hỏi nhiều kiến thức (knowledge-intensive), cần thông tin chính xác, cập nhật, hoặc thuộc một lĩnh vực chuyên biệt (Brown et al., 2020; Ji et al., 2023). Điều này đặc biệt tồn kém trong lĩnh vực pháp luật: một người dùng Việt Nam hỏi “*mẹ chồng tôi mất mà không để lại di chúc thì tài sản có thuộc về tôi không?*” cần một câu trả lời được neo vào đúng văn bản luật đang có hiệu lực, chứ không phải một câu trả lời nghe có vẻ hợp lý nhưng chỉ là đoán mò.

Sinh văn bản tăng cường truy xuất (RAG) (Lewis et al., 2020) giảm thiểu vấn đề này bằng cách điều kiện hóa việc sinh văn bản theo các đoạn được truy xuất từ một kho ngữ liệu ngoài. Tuy nhiên, một pipeline RAG chuẩn truy xuất một số lượng đoạn *cố định* và đưa toàn bộ chúng cho generator một cách vô điều kiện. Điều này dẫn đến hai hệ quả mà chúng tôi quan sát trực tiếp trong thực nghiệm (Mục 6): (i) các đoạn truy xuất không liên quan có thể gây nhiễu hoặc đánh lạc hướng generator, và (ii) generator không có cơ chế nội tại nào để kiểm tra xem câu trả lời của chính nó có thực sự được hỗ trợ bởi những gì đã truy xuất hay không, hay câu trả lời đó có hữu ích hay không – nên nó hoặc “ảo giác” một cách tự tin, hoặc từ chối trả lời một cách không hữu ích.

Self-RAG (Asai et al., 2024) giải quyết cả hai vấn đề trên bằng cách huấn luyện một LM duy nhất để xen kẽ việc sinh văn bản với các *token phản tư* đặc biệt: có cần truy xuất hay không (RETRIEVE), mỗi

đoạn truy xuất có liên quan hay không (ISREL), đoạn văn bản sinh ra có được bằng chứng đã trích dẫn hỗ trợ hay không (ISSUP), và toàn bộ câu trả lời có hữu ích hay không (ISUSE). Tại thời điểm suy luận, các token này điều khiển một quy trình tự phê bình và xếp hạng lại ở cấp độ từng đoạn văn bản (segment-level). Cơ chế này rất thuyết phục, nhưng để tái tạo đúng như mô tả đòi hỏi phải tinh chỉnh cả một critic model (để sinh dữ liệu huấn luyện cho token phản tư) lẫn chính generator trên dữ liệu đã được tăng cường đó – điều không khả thi trong giới hạn tính toán của môi trường Google Colab miễn phí.

Đề án này đặt ra một câu hỏi hẹp hơn, mang tính thực tiễn, được điều chỉnh trực tiếp từ bài báo gốc cho phù hợp với ràng buộc của chúng tôi:

Trên miền văn bản pháp luật tiếng Việt, việc thêm một cơ chế tự phản tư (quyết định truy xuất, đánh giá độ liên quan, đánh giá độ hỗ trợ, đánh giá độ hữu ích) vào RAG có cải thiện độ chính xác truy xuất và chất lượng câu trả lời so với một baseline RAG truy xuất cố định hay không – khi các tín hiệu phản tư được mô phỏng bằng prompting trên một LLM cố định thay vì được học thông qua tinh chỉnh?

Chúng tôi xây dựng ba hệ thống có thể so sánh trực tiếp với nhau – No-RAG, Standard RAG, và một pipeline *Self-RAG-inspired* – trên cùng một retriever và cùng một generator nền, và đánh giá chúng trên VLSP2025 DRiLL (VLSP Organizing Committee, 2025), một bộ dữ liệu shared-task hỏi-đáp pháp luật tiếng Việt thực tế. Đóng góp của chúng tôi gồm: (1) một bản tái tạo chỉ-bằng-prompting, hoạt động được, của bốn tín hiệu phản tư trong Self-RAG, không cần tinh chỉnh; (2) một so sánh định lượng trên một kho ngữ liệu pháp luật tiếng Việt thực tế mà mô hình chưa từng thấy trước đó, cho thấy cải thiện nhất quán đồng thời trên cả độ đúng, độ hỗ trợ và độ hữu ích; (3) một thí nghiệm loại bỏ (ablation) tái hiện được, tách bạch tác động nhân quả của việc lọc liên quan (ISREL) lên tính có-căn-cứ của câu trả lời; và (4) một trình bày trung thực về những chỗ cách tiếp cận chỉ-dùng-prompting vẫn còn thất bại, cùng với các trở ngại kỹ thuật cụ thể (giới hạn tốc độ gọi API, đầu ra judge bị lỗi định dạng, prompt quá khổ) khi vận hành một pipeline LLM-judge ở quy mô lớn trên hạ tầng miễn phí.

Token	Câu hỏi mà token trả lời	Nhãn
RETRIEVE	Có cần truy xuất hay không?	Yes / No / Continue
ISREL	Đoạn này có liên quan không?	Relevant / Irrelevant
ISSUP	Câu trả lời có được bằng chứng hỗ trợ không?	Fully / Partially / No support
ISUSE	Câu trả lời có thực sự hữu ích không?	Useful / Partially / Not useful

Bảng 1: Bốn nhóm token phản tư được Asai et al. (2024) định nghĩa (Mục 3, Bảng 1 của bài báo gốc), mà chúng tôi tái tạo bằng prompting thay vì tinh chỉnh.

2 Related Work

Large Language Models and Hallucination.

Các LLM đã được instruction-tune (Brown et al., 2020) sinh ra văn bản trôi chảy, tự tin ngay cả khi sai sự thật, một dạng lỗi đã được khảo sát kỹ trong Ji et al. (2023). Điều này đặc biệt nguy hiểm đối với hỏi-đáp dựa trên sự kiện (factual QA) trong các lĩnh vực chuyên biệt như pháp luật, nơi người dùng khó có thể tự kiểm chứng một câu trả lời sai nhưng nghe rất tự tin.

Retrieval-Augmented Generation. Lewis et al.

(2020) giới thiệu RAG, kết hợp một retriever dày đặc (dense retriever) với một generator seq2seq được điều kiện hóa theo các đoạn truy xuất, giúp giảm đáng kể hiện tượng ảo giác trên các tác vụ đòi hỏi nhiều kiến thức. Truy xuất dày đặc ở quy mô lớn thường được triển khai bằng tìm kiếm lân cận gần đúng (approximate nearest-neighbour search) trên các chỉ mục embedding như FAISS (Johnson et al., 2019), công cụ mà chúng tôi cũng sử dụng. Tuy nhiên, RAG chuẩn truy xuất một số lượng đoạn cố định bất kể chúng có thực sự liên quan đến câu hỏi hay không, và không có cơ chế nào để kiểm chứng câu trả lời sinh ra so với bằng chứng đã được cung cấp.

Self-RAG. Asai et al. (2024) đề xuất huấn luyện

một LM duy nhất để thích ứng quyết định *khi nào* cần truy xuất và *tự phê bình* chính việc truy xuất và sinh văn bản của nó, bằng cách tinh chỉnh mô hình (dùng một critic model được huấn luyện riêng để giám sát) nhằm sinh ra bốn nhóm *token phản tư*, được tóm tắt ở Bảng 1.

Tại thời điểm suy luận, các token này điều khiển một quy trình truy xuất thích ứng và xếp hạng lại kiểu beam-search ở cấp độ từng đoạn văn bản. Đề án của chúng tôi tái tạo đúng bốn tín hiệu và luồng điều khiển tổng thể tương tự, nhưng tính mỗi tín

hiệu bằng một lời gọi prompting có cấu trúc riêng biệt tới một LLM instruction-following tổng quát, cố định, thay vì tinh chỉnh generator để nó tự sinh ra các token này như một phần của chuỗi token đầu ra. Cách làm này đánh đổi chi phí huấn luyện (thứ mà chúng tôi không có ngân sách tính toán để thực hiện) lấy chi phí gọi API tại thời điểm suy luận – khoảng năm lượt gọi LLM cho mỗi câu hỏi thay vì một – và đánh đổi đặc tính độ tin cậy của một mô hình instruction tổng quát so với một critic được huấn luyện chuyên biệt, cả hai điều này chúng tôi sẽ quay lại ở Mục 7.

3 Methodology

Chúng tôi triển khai và so sánh ba hệ thống dùng chung cùng một retriever và generator nền, chỉ khác nhau ở mức độ truy xuất và tự phản tư được áp dụng:

Hệ 1 – No-RAG:

Câu hỏi → LLM → Câu trả lời

Hệ 2 – Standard RAG:

Câu hỏi → truy xuất top- k điều luật → prompt LLM với (câu hỏi, ngữ cảnh) → Câu trả lời

Hệ 3 – Self-RAG-inspired:

Câu hỏi
→ [RETRIEVE?] quyết định có cần truy xuất hay không
→ truy xuất top- k điều luật (nếu cần)
→ [ISREL] đánh giá độ liên quan của từng điều luật, loại bỏ điều không liên quan
→ sinh câu trả lời từ các bằng chứng còn lại (kèm trích dẫn)
→ [ISSUP] đánh giá câu trả lời có được các trích dẫn hỗ trợ hay không
→ [ISUSE] đánh giá câu trả lời có hữu ích hay không
→ Câu trả lời + báo cáo phản tư

Design of the Reflection Modules. Mỗi trong bốn module là một lượt gọi LLM duy nhất với một prompt có cấu trúc, chỉ yêu cầu JSON; không module nào là một mô hình được huấn luyện riêng. Chúng tôi đưa ra các lựa chọn thiết kế có chủ đích sau, xuất phát từ ràng buộc thực tế của một API LLM miễn phí (Mục 4):

- **[RETRIEVE]** mặc định chọn RETRIEVE khi không parse được đầu ra của mô hình, vì hầu như mọi câu hỏi thật trong bộ dữ liệu đều thực sự cần truy xuất – một thiên lệch an toàn nhằm tránh âm thầm bỏ qua bằng chứng.
- **[ISREL]** đánh giá *toàn bộ* các đoạn đã truy xuất cho một câu hỏi trong một lượt gọi theo lô (batched call, trả về một mảng JSON), thay vì gọi độc lập cho từng đoạn như trong bài báo gốc

– một đánh đổi vì hiệu năng, do gọi theo từng đoạn sẽ nhân ngân sách API vốn đã bị giới hạn tốc độ lên k lần. Các mục không parse được hoặc bị thiếu sẽ mặc định là RELEVANT, cũng theo hướng thiên lệch an toàn là không âm thầm loại bỏ bằng chứng.

- **Bước sinh câu trả lời** dùng một trong hai prompt tùy theo kết quả của các bước trước: một prompt có ngữ cảnh khi vẫn còn bằng chứng liên quan sau khi lọc, hoặc một prompt không-ngữ-cảnh (trả lời từ kiến thức chung và nêu rõ việc không có trích dẫn cụ thể) khi RETRIEVE = No hoặc toàn bộ đoạn truy xuất đã bị ISREL lọc bỏ. Việc chọn prompt thích ứng này chính là điểm khác biệt hành vi thực sự so với Standard RAG, vốn luôn điều kiện hóa theo một ngữ cảnh cố định.
- **[ISSUP]** chỉ được gọi khi còn ít nhất một đoạn liên quan sau khi lọc; nếu không, độ hỗ trợ được gán trực tiếp là NOT_APPLICABLE thay vì yêu cầu judge đánh giá độ hỗ trợ so với bằng chứng không tồn tại.
- **[ISUSE]** luôn được gọi, bất kể có bằng chứng nào được sử dụng hay không, vì độ hữu ích là một thuộc tính của câu trả lời cuối cùng, không phụ thuộc vào cơ sở bằng chứng của nó.

4 Implementation

4.1 Data

Chúng tôi sử dụng **VLSP2025 DRiLL** (Deep Retrieval in the expansive Legal Landscape) (**VLSP Organizing Committee, 2025**), một bộ dữ liệu shared-task pháp luật tiếng Việt thực tế, không phải một kho ngữ liệu đồ chơi (toy corpus). Bộ dữ liệu nền, **VLQA** (Vietnamese Legal Question Answering), được xây dựng và cung cấp bởi nhóm tác giả VLQA (VAC), đại diện bởi TS. Vương Thị Hải Yến (Đại học Công nghệ, Đại học Quốc gia Hà Nội) (**Yến and VLQA Author Consortium (VAC), 2025**); các câu hỏi và câu trả lời chi tiết được VAC tạo ra thông qua một diễn đàn pháp lý, các điều luật hiện hành do chuyên gia pháp lý của VAC cung cấp, và toàn bộ dữ liệu được VAC trực tiếp gán nhãn. Chúng tôi được VAC cấp quyền truy cập và sử dụng dữ liệu VLQA cho mục đích nghiên cứu phi thương mại theo một biên bản quyền sử dụng dữ liệu (data usage memorandum) ký ngày 12/08/2026 giữa VAC và nhóm nghiên cứu.¹ Kho

¹Theo điều khoản của biên bản, mọi công bố sử dụng dữ liệu VLQA phải tham khảo cả kỷ yếu VLSP DRiLL 2025

tri thức gồm 2.157 văn bản pháp luật với tổng cộng 59.636 điều luật, mỗi điều được định danh bằng một mã điều (`aid`) duy nhất toàn cục, đã được kiểm chứng không có trùng lặp trên toàn bộ kho ngữ liệu. Tập có nhãn (`train.json`) gồm 2.190 câu hỏi thật, được diễn đạt theo phong cách khẩu ngữ kiểu “hỏi luật sư”, mỗi câu được gán nhãn với một danh sách `aid` liên quan (trung bình 1.34, tối thiểu 1, tối đa 9, không bao giờ rỗng) và một câu trả lời tư vấn pháp lý do con người viết, dạng tự do. Hai tập `public_test/private_test` chính thức có nhãn bị ẩn và vốn dùng cho bảng xếp hạng của `shared-task`; do hạn nộp hệ thống của VLSP2025 DRiLL (12/08/2025) đã qua từ trước khi đồ án này bắt đầu, hai tập này không thể dùng để đánh giá và bị loại khỏi phạm vi đồ án. Do đó, mọi so sánh trong báo cáo này đều dùng một **tập phát triển (dev set) cố định gồm 250 câu hỏi** được lấy mẫu từ `train.json` (seed 42), được tái sử dụng giống hệt nhau qua mọi notebook và mọi hệ thống để đảm bảo so sánh công bằng trên cùng một tập câu hỏi.

4.2 Retriever

Các điều luật được chia đoạn (`chunk`) bằng một bộ tách ưu tiên theo đoạn văn (`paragraph-first`), dự phòng bằng cửa sổ 800 ký tự chồng lấp 150 ký tự cho những điều luật dài bất thường. Các đoạn được embedding bằng `bkai-foundation-models/vietnamese-bi-encoder` (BKAI Foundation Models, 2023), một sentence encoder chuyên cho tiếng Việt, và được lập chỉ mục bằng FAISS IndexFlatIP (Johnson et al., 2019) để tìm kiếm tích vô hướng (`inner-product`) chính xác. Việc truy xuất và tính điểm luôn được thực hiện ở cấp độ điều luật (`aid`), khớp với độ chi tiết của nhãn vàng (`gold annotation`).

4.3 Generator and Reflection Prompts

Toàn bộ việc sinh câu trả lời và cả bốn module phản tư đều dùng chung một LLM nền, truy cập qua Groq API (Groq, Inc., 2024) (được chọn vì không yêu cầu thiết lập billing, khác với Google Gemini API mà chúng tôi đã thử dùng ban đầu). Model sinh không bị `hardcode`: khi khởi động, pipeline truy vấn danh sách model chat khả dụng của tài khoản và chọn từ một danh sách ứng viên đã xếp hạng sẵn, tự động chuyển sang ứng viên tiếp theo khi `quota` của một model bị cạn. Mỗi prompt phản tư yêu cầu model trả về đúng một đối tượng JSON; phản hồi

(VLSP Organizing Committee, 2025) lần công trình dữ liệu VLQA (Yên and VLQA Author Consortium (VAC), 2025), điều mà báo cáo này tuân thủ.

được parse bằng một bộ trích xuất chịu lỗi (`fault-tolerant`), tự bóc các dấu code-fence Markdown và, nếu cần, dự phòng bằng một chuỗi con JSON trích xuất qua biểu thức chính quy (`regex`), vì các model instruction tổng quát không phải lúc nào cũng trả về JSON thuần túy dù đã được yêu cầu rõ ràng.

4.4 System Interface

Vòng lặp sinh câu trả lời của mỗi hệ thống ghi checkpoint từng dòng JSON xuống đĩa cho mỗi câu hỏi, nên một lượt chạy bị gián đoạn do mất kết nối phiên Colab có thể tiếp tục mà không cần làm lại các câu đã xong. Notebook Self-RAG-inspired còn có thêm một demo tương tác nhỏ: với một vài câu hỏi ví dụ, nó chạy toàn bộ pipeline từ đầu đến cuối và in ra quyết định truy xuất, các trích dẫn sau khi lọc, câu trả lời, và kết quả ISSUP/ISUSE – có thể dùng để demo trực tiếp mà không cần chờ một lượt đánh giá đầy đủ.

4.5 Implementation Challenges

Vận hành ổn định một pipeline LLM-judge trên hạ tầng miễn phí hóa ra là một phần không hề nhỏ trong công sức kỹ thuật, và chúng tôi báo cáo ba lỗi cụ thể đã được chẩn đoán và sửa, vì chúng ảnh hưởng trực tiếp đến mức độ tin cậy của dữ liệu đánh giá (Mục 5):

- Nhiễm vết suy luận (reasoning-trace contamination).** Một trong các model ứng viên của chúng tôi là một model “thinking”, luôn sinh ra một khối suy luận `<think>...</think>` trước phần đầu ra thực sự, ngay cả khi prompt yêu cầu chỉ trả JSON. Lỗi này làm hỏng cả câu trả lời được lưu lẫn mọi lượt gọi judge vô tình rơi vào model này, vốn âm thầm rơi về nhãn mặc định khi parse thất bại – biểu hiện qua việc 100% câu hỏi rơi vào đúng một nhãn cho một vài chỉ số, một điều phi thực tế về mặt thống kê. Chúng tôi đã sửa bằng một bộ lọc `strip_think()` dựa trên `regex`, áp dụng trước cả bước parse JSON lẫn bước lưu câu trả lời, và hạ độ ưu tiên của model này trong thứ tự chuyển đổi (`failover`).
- Âm thầm dùng giá trị mặc định khi hết quota.** Khi toàn bộ model ứng viên cùng hết quota giữa chừng, vòng lặp đánh giá ban đầu vẫn tiếp tục ghi hàng trăm bản ghi judge đã âm thầm rơi về nhãn mặc định, thay vì dừng lại. Chúng tôi đã sửa bằng cách kiểm tra model khả dụng trước mỗi câu hỏi và dừng ngay khi không còn model nào, đồng thời ghi lại lý do (`_reason`) của

từng judge và in ra một cảnh báo rõ ràng – giúp phần dữ liệu còn bị mắc định trở nên có thể nhìn thấy và kiểm chứng được, thay vì âm thầm trôi qua.

3. **Prompt quá khổ bị chặn đoán nhầm thành hết quota.** Một prompt đơn lẻ quá lớn – lượt gọi judge độ hỗ trợ (support-judge), vốn ghép nguyên văn tối đa năm điều luật – thỉnh thoảng vượt giới hạn số token mỗi phút của API. Cách xử lý lỗi ban đầu của chúng tôi coi lỗi này giống hết việc hết quota kéo dài nhiều giờ, khiến toàn bộ model ứng viên bị vô hiệu hóa vĩnh viễn cho phần còn lại của lượt chạy chỉ sau một vài câu hỏi. Chúng tôi đã sửa bằng cách (i) không còn đánh dấu một model là “đã cạn” chỉ vì lỗi cụ thể này, và (ii) cắt bớt mỗi điều luật về một độ dài tối đa cố định trước khi dựng ngữ cảnh bằng chứng, giảm khả năng gặp lại lỗi này ngay từ đầu.

5 Experiments

Research Question. Việc thêm cơ chế tự phản tư (quyết định truy xuất thích ứng, đánh giá độ liên quan, đánh giá độ hỗ trợ, đánh giá độ hữu ích) vào RAG có cải thiện độ chính xác truy xuất và chất lượng câu trả lời trên hỏi-đáp pháp luật tiếng Việt hay không, so với một baseline RAG truy xuất cố định?

Sample Size. Mọi hệ thống đều được đánh giá trên cùng một tập phát triển cố định gồm 250 câu hỏi (Mục 4). Standard RAG hoàn thành cả 250 câu; pipeline Self-RAG-inspired, vốn tốn khoảng năm lượt gọi LLM cho mỗi câu hỏi thay vì một, hoàn thành 202/250 câu (80.8%) trong giới hạn tốc độ của gói miễn phí mà đề án có thể dùng. Bước chấm điểm hậu kiểm (post-hoc) cho correctness/support/usefulness sau đó hoàn thành trên 187 trong số 202 câu đó (92.6%) trước khi cạn cùng ngân sách miễn phí. Chúng tôi báo cáo mọi kết quả chính trên tập con chung, đã được chấm điểm đầy đủ này, với $n = 187$ – một giới hạn cứng do hạ tầng API miễn phí áp đặt, chứ không phải một lựa chọn phương pháp luận, và chúng tôi coi đây là một hạn chế tương minh (Mục 8) thay vì giả vờ rằng toàn bộ 250 câu đã được đánh giá.

Baselines. No-RAG trả lời trực tiếp từ kiến thức tham số (parametric knowledge) của LLM, không có ngữ cảnh truy xuất nào. **Standard RAG** truy xuất cố định top-5 điều luật và điều kiện hóa việc

sinh câu trả lời theo cả năm điều đó một cách vô điều kiện, không lọc và không có bước tự phê bình.

Metrics. Truy xuất được chấm bằng Recall@ k , Precision@ k , và Mean Reciprocal Rank (MRR) so với danh sách `aio` vàng, tính hoàn toàn bằng Python thuần, không qua LLM. *Chất lượng câu trả lời* được một LLM judge chấm (nhiệt độ 0, dùng cùng một prompt cho cả ba hệ thống để đảm bảo công bằng) trên ba trục: **Correctness** (CORRECT / PARTIALLY_CORRECT / INCORRECT, so với câu trả lời vàng do con người viết), **Support** (FULLY_SUPPORTED / PARTIALLY_SUPPORTED / NOT_SUPPORTED / NOT_APPLICABLE khi không dùng bằng chứng nào), và **Usefulness** (USEFUL / PARTIALLY_USEFUL / NOT_USEFUL). Với mỗi trục, chúng tôi báo cáo một *rate* (tỷ lệ câu hỏi đạt đúng nhãn tốt nhất) và một *score* (điểm có trọng số một phần, 1 / 0.5 / 0), vì chỉ riêng rate sẽ gộp chung “đúng một phần” và “sai” vào cùng một nhóm.

Judge Data-Quality Validation. Trước khi tin vào đầu ra của judge, chúng tôi kiểm tra hai dấu hiệu lỗi đã nêu ở Mục 4.5: nhiễm vết suy luận và nhãn mắc định âm thầm. Trên mẫu cuối cùng $n = 187$, mức độ nhiễm không đáng kể (0/187 với No-RAG và Standard RAG, 6/187 = 3.2% với Self-RAG), và chỉ 12/187 dòng (6.4%) có cả sáu lượt gọi judge cùng rơi về mắc định – phù hợp với mức nhiễu thông thường của LLM-judge ở quy mô này hơn là một lỗi hệ thống, và giảm mạnh so với mức 200/202 (99%) quan sát được trước khi áp dụng các bản sửa ở Mục 4.5. Chúng tôi cũng xác nhận rằng phân bố nhãn trên cả ba trục chất lượng thực sự đa dạng ở mọi hệ thống (Mục 6), thay vì dồn hết vào một nhãn chiếm ưu thế duy nhất như trước khi sửa lỗi – đây là bằng chứng mạnh nhất cho thấy judge đang thực sự chấm điểm chứ không phải rơi về mắc định.

6 Results

6.1 Main Comparison

Bảng 2 cho thấy kết quả chính: hệ Self-RAG-inspired vượt trội cả hai baseline trên mọi chỉ số chất lượng câu trả lời cùng lúc – Correctness, Support, và Usefulness – đây là bằng chứng mạnh nhất cho giá trị của cơ chế phản tư.

6.2 Retrieval Quality Before and After ISREL

Bảng 3 tách riêng tác động của bộ lọc liên quan: trước khi lọc, kết quả truy xuất top-5 thô của Self-RAG gần như giống hệt Standard RAG (đúng như

Hệ thống	Recall	Prec.	MRR	Correctness rate / score	Support rate	Usefulness rate / score
No-RAG	–	–	–	16.0% / 0.441	– (N/A)	50.8% / 0.749
Standard RAG	0.497	0.127	0.421	25.7% / 0.444	53.5%	25.7% / 0.497
Self-RAG-inspired	0.425	0.280	0.422	33.2% / 0.551	62.0%	62.6% / 0.791

Bảng 2: So sánh chính giữa ba hệ thống trên mẫu đánh giá chung ($n = 187$). Các chỉ số truy xuất của Self-RAG được tính *sau* khi lọc bằng ISREL (xem Bảng 4). Support không áp dụng được cho No-RAG (không dùng bằng chứng nào).

Cấu hình	Recall	Prec.	MRR
Standard RAG (top-5 cố định)	0.497	0.127	0.421
Self-RAG, trước ISREL	0.503	0.129	–
Self-RAG, sau ISREL	0.425	0.280	0.422

Bảng 3: Tác động của việc lọc bằng ISREL lên chất lượng truy xuất ($n = 187$; “trước” chỉ tính trên 178 câu hỏi có RETRIEVE = Yes). ISREL đánh đổi mức giảm recall tương đối khoảng 15–17% để lấy mức tăng precision khoảng 2.2 lần – tái hiện nhất quán qua ba lần đánh giá độc lập của đồ án.

kỳ vọng, vì cả hai dùng chung một retriever); sau ISREL, precision tăng hơn gấp đôi với cái giá là giảm nhẹ recall.

6.3 Reflection-Signal Distribution

Trên toàn bộ 202 lượt chạy Self-RAG đã hoàn thành, RETRIEVE chọn truy xuất cho 193/202 câu hỏi (95.5%) và bỏ qua truy xuất cho 9/202 câu (4.5%) – gần như toàn bộ là các câu hỏi về chính sách, quy hoạch, hoặc kiến thức chung không cần trích dẫn điều luật (Mục 6, phân tích case study). Trong số các câu trả lời được sinh ra, ISSUP gán nhãn 128/202 (63.4%) là FULLY_SUPPORTED, 35/202 (17.3%) là PARTIALLY_SUPPORTED, 2/202 (1.0%) là NOT_SUPPORTED, và 37/202 (18.3%) là NOT_APPLICABLE (không dùng bằng chứng nào). ISUSE gán nhãn 125/202 (61.9%) là USEFUL, 69/202 (34.2%) là PARTIALLY_USEFUL, và 8/202 (4.0%) là NOT_USEFUL.

6.4 Ablation: Does ISREL Filtering Causally Improve Support?

Bảng 4, theo chúng tôi, là kết quả quan trọng nhất của đồ án: khi ISREL thực sự loại bỏ ít nhất một đoạn gây nhiễu, xác suất câu trả lời được các trích dẫn của nó hỗ trợ đầy đủ tăng hơn gấp đôi. Trung bình, 5.00 điều luật được truy xuất cho mỗi câu hỏi và còn lại 2.39 điều sau khi lọc (~52% được giữ lại); 152/193 câu hỏi có RETRIEVE = Yes (78.8%) có ít nhất một đoạn bị lọc bỏ.

Kết quả ISREL	n	Được hỗ trợ đầy đủ	Tỷ lệ
Không lọc đoạn nào	41	16 / 41	39.0%
≥ 1 đoạn bị lọc	152	112 / 124	90.3%

Bảng 4: Tỷ lệ FULLY_SUPPORTED (không tính NOT_APPLICABLE) theo việc ISREL có loại bỏ ít nhất một đoạn truy xuất hay không. Tái hiện ba lần qua các lần đánh giá độc lập của đồ án (15%→83.2%; 39.0%→90.3%; 39.0%→90.3%) – phát hiện đáng tin cậy nhất của chúng tôi.

6.5 Case Studies

Standard RAG’s Honest-but-Unhelpful Refusal (qid 885). Với câu hỏi về quyền thừa kế của con dâu khi bố mẹ chồng mất mà không để lại di chúc, Standard RAG – được điều kiện hóa theo cả năm điều luật truy xuất được (phần lớn không liên quan) – trả lời: “*Dựa trên các điều luật được cung cấp, không đủ căn cứ để trả lời...*”, bị chấm INCORRECT vì Bộ luật Dân sự thực ra có quy định rõ con dâu không thuộc hàng thừa kế theo pháp luật. Self-RAG, sau khi lọc bằng ISREL, trả lời trực tiếp và đúng, trích dẫn đúng điều luật liên quan, và được chấm CORRECT cùng FULLY_SUPPORTED.

Noise Drowning Out the Correct Citation (qid 1027). Với câu hỏi về số lượng tài khoản chứng khoán mà một công ty quản lý quỹ nước ngoài được phép mở tại Việt Nam, năm điều luật được truy xuất (aid 8149, 56180, 8311, 54667, 8314), trong đó ISREL chỉ giữ lại đúng điều luật thực sự liên quan (8149). Standard RAG, dùng cả năm điều, khẳng định không có thông tin nào về việc này trong các điều luật được cung cấp (INCORRECT); Self-RAG, chỉ dùng điều luật đã lọc, trả lời đúng “02 tài khoản” kèm trích dẫn rõ ràng (CORRECT, FULLY_SUPPORTED). Ở cả hai qid 885 và qid 1027, mô hình lỗi giống nhau: các điều luật không liên quan trong ngữ cảnh cố định của Standard RAG che khuất điều luật thực sự liên quan, khiến generator kém tự tin hoặc trả lời sai, trong khi việc lọc bằng ISREL loại bỏ được nhiễu đó.

A Genuine Self-RAG Hallucination (qid 1179).

Với câu hỏi về cơ quan nào có thẩm quyền cho thuê đất để xây trụ sở đại sứ quán nước ngoài, Self-RAG trả lời “Bộ Xây dựng”, trích dẫn một điều luật cụ thể nhưng sai; cơ quan đúng, theo lý giải của judge, là Ủy ban nhân dân cấp tỉnh, thuộc một điều luật khác. Câu trả lời này bị chấm INCORRECT dù có trích dẫn đúng định dạng – một trường hợp ảo giác thật sự, cho thấy việc ISSUP kiểm tra trích dẫn có đúng định dạng hay không không đảm bảo rằng nội dung được trích dẫn đã được generator hiểu đúng.

A Retrieval Miss (qid 15). Với câu hỏi về trách nhiệm hình sự khi mua bán hoặc cho thuê tài khoản ngân hàng, Self-RAG trả lời rằng không tìm thấy quy định liên quan, trong khi câu trả lời vàng có trích dẫn mức phạt cụ thể. Do điều luật đúng chưa bao giờ nằm trong top-5 kết quả truy xuất ngay từ đầu, không có bước phản tư nào ở phía sau có thể khôi phục lại được – đây là lỗi ở tầng truy xuất, không phải lỗi ở tầng phản tư.

7 Discussion

Why the Improvement Happens. Mô hình lỗi nhất quán xuất hiện trong các case study của chúng tôi (Mục 6) là: một ngữ cảnh top-5 cố định thường chứa các điều luật có liên quan về mặt chủ đề nhưng không thực sự liên quan đến câu hỏi cụ thể, và những điều luật gây nhiễu này khiến generator hoặc né tránh một cách không hữu ích, hoặc trả lời tự tin nhưng sai. ISREL loại bỏ nhiễu này *trước* khi sinh câu trả lời, đó là lý do vì sao độ đúng và tính có-căn-cứ cùng cải thiện thay vì đánh đổi lẫn nhau.

The Standard RAG Usefulness Paradox. Một kết quả đáng chú ý trong Bảng 2 là tỷ lệ Usefulness của Standard RAG (25.7%) lại *thấp hơn* cả No-RAG (50.8%), dù tỷ lệ Correctness của nó cao hơn. Các case study giải thích vì sao: khi được cung cấp năm điều luật cố định, thường không liên quan, Standard RAG trung thực tuân theo chỉ dẫn của nó và từ chối trả lời (“không đủ căn cứ”), điều này bị judge chấm là không hữu ích dù không sai; No-RAG, không bị ràng buộc bởi bằng chứng nào, trả lời tự tin từ kiến thức tham số và được chấm là hữu ích hơn dù kém chính xác hơn. Self-RAG giải quyết được đánh đổi giữa độ đúng và độ hữu ích này: vì ISREL lọc bỏ nhiễu trước khi sinh câu trả lời, khi hệ thống thực sự trả lời, nó trả lời tự tin và chính xác hơn hẳn Standard RAG, đạt được đồng thời cả độ đúng và độ hữu ích cao nhất trong ba hệ thống. Theo chúng tôi, đây là bằng chứng rõ ràng nhất

trong kết quả của đề án cho giá trị thực tiễn của tự phản tư, vượt ra ngoài một con số độ chính xác đơn thuần.

When the System Still Fails. Ba dạng thất bại cụ thể vẫn còn tồn tại, minh họa ở Mục 6: (i) ảo giác với trích dẫn đúng định dạng nhưng sai sự thật (qid 1179), cho thấy việc ISSUP kiểm tra trích dẫn có đúng định dạng không phải là kiểm tra việc hiểu đúng nội dung trích dẫn đó; (ii) đôi khi quyết định RETRIEVE = No hơi vội vàng với những câu hỏi biên, được diễn đạt như mang tính tổ chức hoặc chung chung nhưng thực ra vẫn có quy định pháp luật điều chỉnh; và (iii) các trường hợp truy xuất bị bỏ sót mà không bước phản tư nào phía sau có thể sửa được, vì tự phê bình chỉ có thể đánh giá bằng chứng đã được cung cấp cho nó, chứ không thể khôi phục bằng chứng chưa từng được truy xuất.

Cost and Operational Limitations. Cách tiếp cận chỉ-dùng-prompting đánh đổi chi phí tính chính lấy chi phí tại thời điểm suy luận: pipeline Self-RAG-inspired tốn khoảng năm lượt gọi LLM cho mỗi câu hỏi (quyết định truy xuất, đánh giá liên quan theo lô, sinh câu trả lời, đánh giá độ hỗ trợ, đánh giá độ hữu ích) so với một lượt của Standard RAG, và điều này – kết hợp với giới hạn tốc độ và giới hạn token-mỗi-phút của gói miễn phí – chính là nguyên nhân giới hạn mẫu đánh giá khả dụng của chúng tôi ở mức 187/250 câu hỏi (Mục 5) và đòi hỏi các bản sửa kỹ thuật mô tả ở Mục 4.5. Một mô hình Self-RAG đã được tinh chỉnh, như trong bài báo gốc, sẽ chịu chi phí này một lần trong lúc huấn luyện rồi chỉ tốn một lượt forward pass duy nhất khi suy luận; cách tiếp cận của chúng tôi phải trả một chi phí suy luận lặp lại, lớn hơn, để đổi lấy việc không cần dữ liệu huấn luyện hay ngân sách GPU nào cả.

8 Conclusion

Chúng tôi đã xây dựng một hệ thống hỏi-đáp pháp luật tiếng Việt lấy cảm hứng từ Self-RAG, tái tạo bốn tín hiệu phản tư của Asai et al. (2024) – truy xuất thích ứng, đánh giá độ liên quan, đánh giá độ hỗ trợ, và đánh giá độ hữu ích – hoàn toàn thông qua prompting có cấu trúc trên một LLM cố định, không cần tinh chỉnh. Trên một benchmark pháp luật tiếng Việt thực tế mà mô hình chưa từng thấy trước đó, hệ thống này vượt trội cả baseline No-RAG lẫn baseline Standard RAG top-5 cố định trên mọi chỉ số chất lượng câu trả lời cùng lúc, và bộ lọc liên quan của nó giúp tăng hơn gấp đôi tỷ lệ câu trả

lỗi được hỗ trợ đầy đủ, một phát hiện được tái hiện qua ba lần đánh giá độc lập. Hệ thống cũng giải quyết được một đánh đổi giữa độ đúng và độ hữu ích mà baseline truy xuất cố định gặp phải, bằng cách lọc bỏ bằng chứng gây nhiễu trước khi sinh câu trả lời thay vì sau đó. Những kết quả này cho thấy ý tưởng cốt lõi đằng sau Self-RAG – truy xuất thích ứng và tự phê bình – vẫn có giá trị ngay cả ở dạng hoàn toàn dựa trên prompting, không tinh chỉnh, trên một ngôn ngữ ít tài nguyên và một lĩnh vực chuyên biệt. Hướng phát triển tiếp theo bao gồm: tinh chỉnh một critic model tiếng Việt nhẹ để thay thế các judge dựa trên prompting (giảm cả chi phí lẫn giới hạn cứng về cỡ mẫu do gói miễn phí gây ra trong đồ án này), đánh giá trên một mẫu lớn hơn, và khám phá cơ chế phản tư ở cấp độ token ngay trong lúc sinh văn bản, gần hơn với cơ chế beam-search cấp-đoạn của bài báo gốc, thay vì chấm điểm hậu kiểm (post-hoc) như trong đồ án này.

Limitations

Đồ án này mô phỏng token phản tư bằng cách prompting một LLM instruction-tuned tổng quát, cố định, thay vì tinh chỉnh một critic được huấn luyện chuyên biệt như trong bài báo Self-RAG gốc; các tín hiệu thu được có thể kém hiệu chỉnh (calibrated) và kém nhất quán hơn so với một mô hình được huấn luyện riêng để sinh ra chúng. Chúng tôi dựa vào một LLM judge, thay vì người gán nhãn, để chấm điểm correctness, support, và usefulness thống nhất cho cả ba hệ thống; dù đã kiểm chứng rằng phân bố nhãn của judge đa dạng và không suy biến (Mục 5), vẫn có khoảng 6.4% số dòng được chấm cho thấy dấu hiệu nhiễu do nhãn mặc định ngay cả sau các bản sửa tăng độ ổn định, và không có đánh giá của con người độc lập nào được thực hiện để hiệu chỉnh chính judge đó. Kết quả chính của chúng tôi được tính trên 187 trong số 250 câu hỏi dự kiến của tập phát triển, một giới hạn cứng do giới hạn tốc độ API của gói miễn phí gây ra, chứ không phải một lựa chọn phương pháp luận. Chúng tôi chỉ đánh giá trên một lĩnh vực (hỏi-đáp pháp luật tiếng Việt) và một ngôn ngữ; khả năng khái quát hóa sang lĩnh vực hoặc ngôn ngữ khác chưa được kiểm chứng. Cuối cùng, một số lỗi xuất phát hoàn toàn từ tầng truy xuất (điều luật vàng chưa bao giờ nằm trong top- k kết quả truy xuất), điều mà không module phản tư nào ở phía sau tầng truy xuất có thể sửa được – tự phê bình chỉ giới hạn ở việc generator làm gì với bằng chứng nó nhận

được, chứ không phải nó nhận được bằng chứng nào ngay từ đầu.

Acknowledgments

Đồ án này được thực hiện như sản phẩm cuối kỳ cho học phần CS2308.CH203 – Chuyên đề nghiên cứu và ứng dụng về Xử lý ngôn ngữ tự nhiên, dựa trên Asai et al. (2024). Chúng tôi xin cảm ơn TS. Nguyễn Văn Kiệt đã hướng dẫn và giới thiệu nhóm tác giả VLQA cho đồ án này. Chúng tôi cũng xin cảm ơn nhóm tác giả VLQA (VAC), đại diện bởi TS. Vương Thị Hải Yến (Đại học Công nghệ, Đại học Quốc gia Hà Nội), đã cấp quyền sử dụng bộ dữ liệu VLQA (Yến and VLQA Author Consortium (VAC), 2025) phục vụ nghiên cứu trong khuôn khổ shared-task VLSP2025 DRiLL (VLSP Organizing Committee, 2025).

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. *Self-RAG: Learning to retrieve, generate, and critique through self-reflection*. In *International Conference on Learning Representations (ICLR)*.
- BKAI Foundation Models. 2023. Vietnamese bi-encoder: bkai-foundation-models/vietnamese-bi-encoder. <https://huggingface.co/bkai-foundation-models/vietnamese-bi-encoder>.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901.
- Groq, Inc. 2024. Groq API: LPU inference engine. <https://groq.com>.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel,

Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.

VLSP Organizing Committee. 2025. VLSP 2025 shared task: DRiLL – deep retrieval in the expansive legal landscape. <https://vlsp.org.vn/vlsp2025/eval/drill>. Accessed 2026.

Vương Thị Hải Yến and VLQA Author Consortium (VAC). 2025. VLQA: Vietnamese legal question answering dataset. University of Engineering and Technology, Vietnam National University, Hanoi. Dataset accessed by the authors under a signed data-usage memorandum with the VLQA Author Consortium (VAC), 12 August 2026.